

# 统一建模语言UML

UML由四个部分组成：

基础结构：给出了用于定义建模语言的核心构造物

上层结构：定义了建模语言--URL

对象约束语言：描述以上三种结构中的查询表达式和约束

图交换：图交换规定了如何定义数据交换的XML文件的格式

模型图在逻辑上分为两大类：结构图和行为图（理解记忆，每种逻辑图记几个例子）

结构图(用于系统的静态建模)：分为类图，组合结构图，构件图，部署图，对象图，包图，外扩图

行为图(用于系统的动态建模)：分为活动图，用况图，状态机图，交互图，顺序图，交互概览图，通信图，顶视图

用况图：主要包括四个方面的内容：系统边界，参与者，用况，关系（关联，泛华，包含，扩展）

类图：类图描述了系统中各对象的类型和它们之间的关系（继承，关联，聚合/组合，依赖）

顺序图/通信图：

交互图用于描述对象之间的动态合作关系以及合作过程中的行为次序，包括以下两种：

顺序图：强调按时间顺序的对象间动态交互关系

协作图：强调接受和发型消息的对象的结构组织

顺序图的组成：对象，消息，生命线，激活

通信图的组成：对象，链接，链接上的信息

活动图：活动图描述了在一个过程中，顺序的/并行的活动及其之间的关系

组成元素：动作和活动，活动图的开始，结束，控制流，分支，分叉和汇合，对象流，泳道

状态图：描述单个对象的动态行为

包图：用来描述包及其关系的图，通常用来描述系统，子系统的宏观组成和结构

构件图/部署图：

物理建模：描述组件和节点之间的关系

构件图（组件图），部署图（配件图）

相同的构成元素：

构建，接口，构件实例

构建向外界提供的服务（实现接口）

构建向外界要求的服务（依赖接口）

不同点：

1. 构件图展现构件类型的定义；部署图表现构建实例

2. 构件图偏向于描述构件之间的相互依赖；部署图偏向描述构件在节点中中的运行时的

状态，描述构建运行的环境。

## 面向对象分析

1. OOA模型：就是通过面向对象的分析所建立的系统分析模型，表达了在OOA阶段所认识到的系统成分及彼此之间的关系。

2. （简答题）问题域：被开发系统的应用领域，即在现实世界中这个系统所涉及的业务范围。

系统责任：被开发系统应该具备的职能。

3. 每个图的作用（见上一节）

## 用况图（需求模型）

1. 参与者：参与者的标准图符是一个“人型符号”，参与者的名字放在图符的下方。

识别参与者：人员，外部系统，设备

2. 把用况表示成一个包含用况名字的椭圆。

3. 参与者和用况之间的交互关系称为关联。

4. 用况之间的关系：包含，扩展，继承。（理解记忆）

包含（**include**，虚线箭头）：可以把重复的交互行为放在一个用况中，原有的用况（基用况）再引入该用况，这样就在用况间建立了包含关系《**include**》标记。具有包含关系的用况间并不是一对一的

扩展（**extends**，虚线箭头）：《**extends**》标记

继承：用一个指向一般情况的带有封闭的空心箭头的实线来表示用况之间的继承关系

5. 包含和扩展可以互换吗，那一个更好？（理解记忆）

包含关系表示一个用况包含了另一个用况的功能，被包含的用况是必需的一部分。扩展用况对主用况是可选的，并且它们的触发条件可能不同。这两种关系在用况图中用于不同的目的，无法互换使用。选择使用包含或扩展关系取决于用况之间的实际需求和关系。

用况图：（符号记住，要会画图）

参与者：图符：一个”人形符号“，名字放在图符的下方

三个参与者：人员，外部系统，设备

用况：把用况表示成一个包含用况名字的椭圆

把参与者和用况之间的关联表示成参与这和用况之间的一条实线。若要明确地指出参与者和用况之间的通信是单向的，就在接受通信的那端的关键线上加一个箭头，用以表示指示方向。

包含（**include**，虚线箭头）：可以把重复的交互行为放在一个用况中，原有的用况（基用况）再引入该用况，这样就在用况间建立了包含关系《**include**》标记。具有包含关系的用况间并不是一对一的

扩展（**extends**，虚线箭头）：《**extends**》标记

继承：用一个指向一般情况的带有封闭的空心箭头的实线来表示用况之间的继承关系

可以选择把用况用一个矩形围起来，用来表示系统或子系统的边界

## 类图（基本模型）

1. 类图之间的关系：

继承（在UML中称为泛化）：

关联（一组类或一个类的对象笛卡尔积的一个子集，刻画同种或者异种类别事物间的关系）

依赖：表明一个元素（源元素）的定义或实现依赖另一个元素（被依赖元素）

聚合（整体没有，部分可以存在）/组合（同生同灭）：聚合是表示整体的类和表示部分的类之间“整体-部分”关系。组合是聚合的异种形式，具体就是：在一个类中有一个成员恰好是另一个类的对象

2. 类图的简化：

一是从一般类划分出太多的特殊类，使系统中类的设置太多，增加了系统的复杂性。

二是建立过深的继承层次，增加了系统的理解难度和处理开销

3. 静态关系和动态关系：

静态关系（用一个标识记录）：是指一个事物通过记录另一个事物的标识可以访问对方所形成的关系

动态关系（行为依赖）：即对象之间在行为（操作）上的依赖关系。

4. 接口：声明了一组操作，用以刻画模型元素对外提供的服务或者它所需要的外部接口

类图是一个方框，里面分为三部分：类名，属性名，对象的操作，继承的描述方式和用况图的继承方式一样（用空心箭头）

关联：把二元关联表示成连接两个类符号的实线（为了区别某些东西也可以画成折线），表示方向的时候可以在某一端加上箭头（类似与数据库ER图中的数量对应的关系）

聚合/组合表示：把聚合表示成一条一端带有一个菱形的线段，菱形指向聚集的那一端。如果是组合，菱形是黑心的。

依赖：表示方法类似于用况图中的扩展和包含，虚线箭头，箭头尾部的模型元素（源元素）依赖箭头头部的模型元素（被依赖元素）

## 顺序图（辅助模型）

1. 顺序图和后面的通信图都用于详细描述对象之间的交互

2. 顺序图是一种详细描述对象之间以及对象与参与者之间交互的图，它有一组相互协作的对象或参与者实例以及它们之间发送的消息组成。顺序图中有对象（参与者），消息，生命线和执行规约。

对象生命线：表示对象在一段时间内的存在

执行规约：是一个对象执行一个操作的时期

同步和异步消息

可选执行（**opt**），条件执行（**alt**），并行执行（**par**），循环执行（**loop**）

3. 什么是通信图，和顺序图的区别是什么，分别应用在什么场景

通信图：通信图展示对象之间的消息传递关系和交互流程，通过展示对象之间的关系和消息的传递路径来描述系统的动态行为。

顺序图是一种详细描述对象之间以及对象与参与者之间交互的图，它有一组相互协作的对象或参与者实例以及它们之间发送的消息组成。

通信图适用场景：

描述系统中多个对象之间的协作关系和消息传递路径。

展示对象之间的交互流程和合作行为。

强调对象之间的关联和通信路径。

顺序图适用场景：

描述对象之间的时序关系和消息的传递顺序。

强调对象之间的交互顺序和时序。

描述对象的内部行为和活动。

4. 对象流：包括对象，输入和输出

5. 泳道：一个活动图可以分为不同组，每个组分别称为一个泳道

6. 包之间的关联关系，如何划分组织包？（简单记忆）

包和包之间的关联关系：访问依赖和引入依赖。

如何划分和组织包：良好的包是高内聚的，包之间是松耦合的。

基本策略：识别低层包，合并或组织包，组织包的层次，标识包中的模型元素的可见性，建立包之间的关系。

对象生命线：在图的顶部放置所有的消息开始前就存在的对象，在所有得消息执行后仍然存在的对象，其生命线要延伸超出图中的最后一个箭线。

如果一个对象创建，就把创建对象的箭线的头部画在对象符号上

如果对象在途中销毁，就用一个**x**标记它

执行规约

## 构件

P(156)

## 设计模式

设计模式一共分为三类：

1. 结构型：该类模式通过用接口将实现与抽象联系起来的方式把已有对象组合起来进行建模。

2. 行为型：该类模式通过对变化进行封装使得所建立的模型可以提供灵活的行为方式

3. 创建型：该种模式用于对创建对象建模

- 1.外观模式：外观模式用于为子系统中的一组对象提供统一接口，使得系统更易于使用（继承接口）（P166）  
意图：通过定义所需要的接口，简化原有系统的使用方式  
问题：只使用原系统的部分功能，或以特殊的方式与系统交互  
解决方案：为原有系统的用户提供新的接口
- 2.适配器模式：适配器模式用于将类的一个接口转换为另一个类所需的接口，以使由于接口不兼容的类进行合作（解决互不兼容的类可以进行合作，P168）  
意图：使一个类的接口与一个结构（通常是使用多态机制的继承结构）中的类的接口相匹配  
问题：一个类的操作的语义与一个结构中的类的接口相一致，但特征标记有差异  
解决方案：该模式提供了起包装作用的类，以使得接口相一致
- 3.策略模式：描述了怎样按需要在—组可替换的算法中选用算法，即把所用的一些算法各自封装起来，可根据客户的需要分别使用它们。（类似于算法中的算法库，根据需要直接进行调库，P170）  
意图：根据语境使用不同的业务规则或算法  
问题：处理客户数据的算法种类可能发生变化，且要根据需要选择算法  
解决方案：把对算法的选择和算法的实现相分离，并根据语境选择算法
- 4.观察者模式：用于定义对象间的一对多的依赖关系，当一个对象发生变化并对外发布消息时，所有依赖它的对象都将得到通知并可进行更新。（用于依赖于—对多的关系），P171  
问题：当一个对象发生变化并对外发布消息时，需要向对象数目不定的对象集发出通知  
解决方案：观察者将检测事件的负责委托给一个专门对象
- 5.抽象工厂模式：为创建—组相关或相互依赖的对象提供一个接口，而不需要指出用于创建对象的具体类。  
意图：为特定的用户提供—组对象。P173  
问题：需要实例化出—组相关对象  
解决方案：提供异种方式，把创建对象的规则从使用这些对象的客户对象中提取出来，放在负责创建对象组的工厂对象中。
- 6.工厂方法模式：在—般类中定义—个用于创建对象的接口，让特殊类的对象决定实例化哪一个类，该模式把创建对象的工作推迟给特殊类的对象。  
意图：定义—个用于创建对象的接口，让特殊类的对象创建对象  
问题：需要创建不同类的对象，但不知道创建哪一种，而让特殊类的对象去决策（让特殊类的对象去创建对象）

## 一些预测

简答题：

- 1.UML统一建模语言图，包括哪些图，图的分类（P12），有哪些内容  
答：模型图在逻辑上分为两大类：结构图和行为图（理解记忆，每种逻辑图记几个例子）  
结构图(用于系统的静态建模)：分为类图，组合结构图，构件图，部署图，对象图，包图，外扩图  
行为图(用于系统的动态建模)：分为活动图，用况图，状态机图，交互图，顺序图，交互概览图，通信图，顶视图
- 2.什么事问题域，系统责任  
答：问题域：被开发系统的应用领域，即在现实世界中这个系统所涉及的业务范围。  
系统责任：被开发系统应该具备的职能。
- 3.给一个系统功能，问有哪几个类；如何实现类，怎样识别属性和方法以及它的操作，类图之间的四种关系（继承，关联，依赖，聚合/组合）
- 4.包图（文件夹），以及可见性，包之间的关联关系，如何划分组织包？  
包和包之间的关联关系：访问依赖和引入依赖。  
如何划分和组织包：良好的包是高内聚的，包之间是松耦合的。  
基本策略：识别低层包，合并或组织包，组织包的层次，标识包中的模型元素的可见性，建立包之间的关系
- 5.如何识别控制流，控制流如何同步？  
识别控制流：OOA中定义的主流对象；系统分布方案所要求的多控制流；系统的并发需求所要求的多控制流；为实现方便设立的控制流；根据任务的紧急程度设置控制流；为处理异常事件所设置的控制流；为实现并行计算所设置的控制流；设置起协调者作用的控制流。  
控制流同步：在每一种都对类中定义的操作附加—个同步标记，三种方法：Sequential（顺序的），Guarded（受监护的），Concurrent（并发的）
- 6.什么是持久对象：

持久对象是指在程序的不同执行周期中保持其状态和数据的对象。持久对象通常与持久化存储（如数据库）相关联，可以在不同的程序执行周期中进行存储和检索。

7.OOD的评价准则，什么是耦合，什么是内聚；

OOD的评价准则：单一责任原则，开放-封闭原则，里氏替换原则，依赖倒置原则，接口隔离原则，迪米特法则

耦合：耦合是指一个软件系统中各个模块之间相互依赖的程度。耦合度越低，模块之间的依赖关系越小，代码的复用性、可维护性和可测试性越好。低耦合意味着一个模块的修改不会对其他模块产生太大的影响。

内聚：内聚是指一个模块内部各个元素彼此之间相关性的程度。高内聚意味着模块内部的元素之间关联紧密，功能相近的代码被组织在一起，模块的功能性和可读性提高。

8.如何设计人机交互部分，以及注意事项。

如何设计：1.分析与系统交互的人员参与者

2.从用况分析人机交互

3.设计输入输出

4.设计命令的组织

5.用OO概念表达所有的界面成分

6.衔接界面模型和问题域模型

注意事项：1.易学，易用，操作方便

2.尽量保持一致性

3.及时提供有意义的反馈

4.减少输入输出操作

5.放置灾难性的错误

9.设计模式的一些相关内容（可能会考）

问答题：

1.给要求题目，画图；

2.将一个设计模式改成另一个设计模式；

分析题：

1.给一个场景，问用什么设计模式合适

2.一个场景，供需关系，平台功能，画用例图，类图，简要分析。状态图和时序图

4.画相应的类图和实现的功能，类的可见性，操作，以及它如何表达

5.给一个构件图，说明它的含义；给出形成的关联关系，要求会画构件

设计题：

1.给一个案例，列出一些功能（包括系统级功能和不同用户的功能），设计类图和用况图，针对某一部分功能过程画一个泳道

必考的内容：

用况图，类图，顺序图（画图），活动图（分析或问答），状态机图，包图（简答），6个设计模式（三种类型：结构型，行为型，创建型，考三个）

人机交互：

控制驱动

